课后作业：运动学验证与分析

环境：Ubuntu + ROS + Gazebo

ROS 工作空间：~/ws_gazebo，功能包：humanoid_sim

基础作业

1.1 正运动学验证

- 启动仿真，使用 arm_kinematics.py fk 和 tf_echo /torso /left_hand 验证正运动学公式。简要给出 3 组关节角，分别发送关节命令，并提交测试结果表格；
- 分析为什么现在输入 q2 为正值时，FK 脚本输出与 tf_echo 结果不一致？提交思考结果；
- 针对不一致的问题，你会如何修改代码，请给出修改后的代码局部细节和运行结果截图。

1.2 逆运动学闭环验证

- 使用 arm_kinematics.py ik 指定末端目标位置，求解 IK，将输出的 rostopic pub 命令发送给机器人，用 tf_echo 验证手端是否到达目标，提交测试表格；
- 分析 ik 的输入参数，即输入的末端位置和 torso 中显示的为什么不一致？（提示：torso 坐标系和肩部坐标系）给出思考结果，并分析解决思路。

基础作业提交要求

将以上内容整理为 **一份 PDF**，包含：

- 验证测试表格
- 思考题回答
- Gazebo 机器人姿态截图（至少 1 张）
- tf_echo 终端输出截图（至少 2 张）

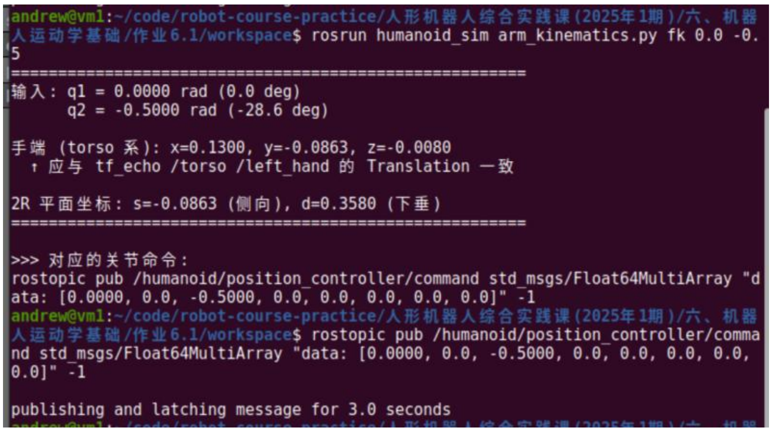
Answers：

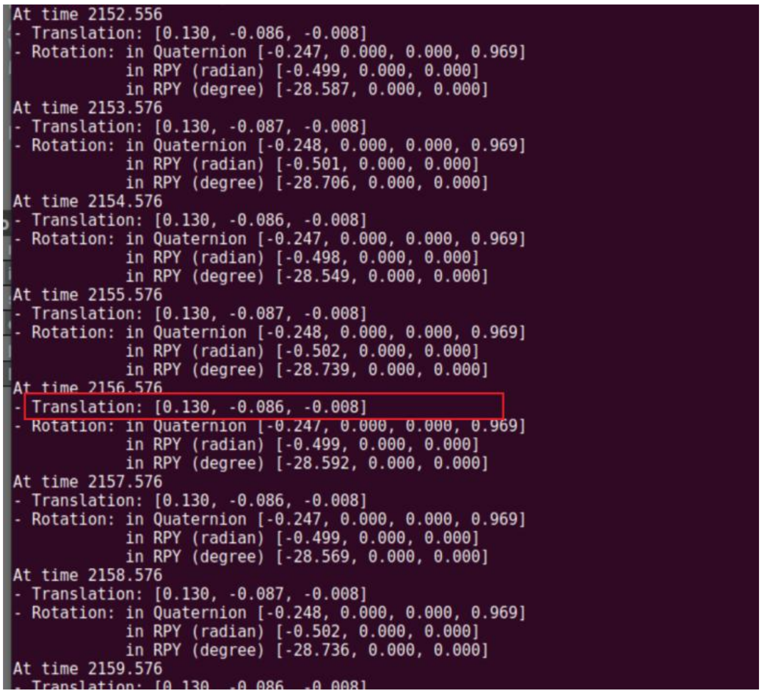
1.1 正运动学验证

（1）测试结果表格

序号	q1	q2	脚本预计算	tf_echo /torso /left_hand 输出
1	0.8	-0.9	x=0.1300, y=0.1255, z=0.0316	0.130, 0.126, 0.032
2	-0.8	-0.9	x=0.1300, y=-0.3220, z=0.2339	0.130, -0.322, 0.233
3	0.0	-0.5	x=0.1300, y=-0.0863, z=-0.0080	0.130, -0.086, -0.008

测试截图:





（2）思考题

q2 代表左肘关节转动角，在 urdf 文件中限制范围[-2.0, 0]。当 q2 输入正值后，会被修正为 0，也就是相对于左上臂无转动，导致 FK 脚本输出与 tf_echo 结果不一致。

```xml
<joint name="left_elbow" type="revolute">
  <parent link="left_upper_arm"/>
  <child link="left_forearm"/>
  <origin xyz="0 0 -0.2" rpy="0 0 0"/>
  <axis xyz="1 0 0"/>
  <limit lower="-2.0" upper="0" effort="10" velocity="1.0"/>
</joint>
```

可以修改 urdf 中的 limit 属性或者完善 arm_kinematics.py。

1. 修改 urdf 中的 limit 属性

```xml
  <limit lower="-2.0" upper="2.0" effort="10" velocity="1.0"/>
```

运行结果:

```
andrew@vm1:~/code/robot-course-practice/人形机器人综合实践课(2025年1期)/六、机器人运动学基础/作业6.1/workspace$ rosrun humanoid_sim arm_kinematics.py fk 0.5 0.5
============================================================
输入: q1 = 0.5000 rad (28.6 deg)
      q2 = 0.5000 rad (28.6 deg)

手端 (torso 系): x=0.1300, y=0.2473, z=0.0772
  ↑ 应与 tf_echo /torso /left_hand 的 Translation 一致

2R 平面坐标: s=0.2473 (侧向), d=0.2728 (下垂)
============================================================

>>> 对应的关节命令:
rostopic pub /humanoid/position_controller/command std_msgs/Float64MultiArray "data: [0.5000, 0.0, 0.5000, 0.0, 0.0, 0.0, 0.0, 0.0]" -1
andrew@vm1:~/code/robot-course-practice/人形机器人综合实践课(2025年1期)/六、机器人运动学基础/作业6.1/workspace$ rostopic pub /humanoid/position_controller/command std_msgs/Float64MultiArray "data: [0.5000, 0.0, 0.5000, 0.0, 0.0, 0.0, 0.0, 0.0]" -1
publishing and latching message for 3.0 seconds
```

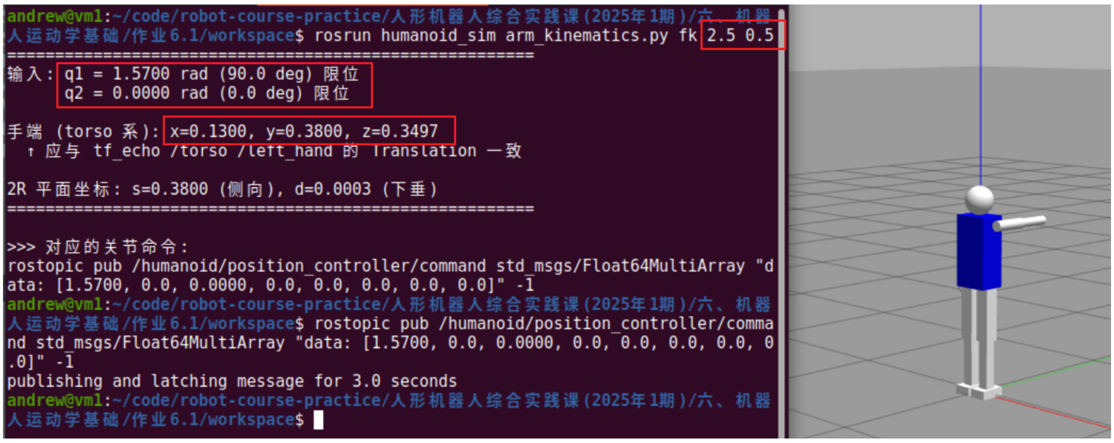
```
At time 115.714
- Translation: [0.130, 0.247, 0.077]
- Rotation: in Quaternion [0.478, 0.000, 0.000, 0.878]
            in RPY (radian) [0.998, -0.000, 0.000]
            in RPY (degree) [57.156, -0.000, 0.000]
At time 116.714
- Translation: [0.130, 0.247, 0.077]
- Rotation: in Quaternion [0.479, 0.000, 0.000, 0.878]
            in RPY (radian) [0.998, -0.000, 0.000]
            in RPY (degree) [57.188, -0.000, 0.000]
At time 117.714
- Translation: [0.130, 0.248, 0.078]
- Rotation: in Quaternion [0.480, 0.000, 0.000, 0.877]
            in RPY (radian) [1.002, -0.000, 0.000]
            in RPY (degree) [57.424, -0.000, 0.000]
At time 118.714
- Translation: [0.130, 0.248, 0.077]
- Rotation: in Quaternion [0.480, 0.000, 0.000, 0.877]
            in RPY (radian) [1.001, -0.000, 0.000]
            in RPY (degree) [57.343, -0.000, 0.000]
At time 119.714
- Translation: [0.130, 0.247, 0.077]
- Rotation: in Quaternion [0.479, 0.000, 0.000, 0.878]
            in RPY (radian) [1.000, -0.000, 0.000]
            in RPY (degree) [57.287, -0.000, 0.000]
At time 120.714
- Translation: [0.130, 0.247, 0.077]
- Rotation: in Quaternion [0.479, 0.000, 0.000, 0.878]
            in RPY (radian) [0.999, -0.000, 0.000]
            in RPY (degree) [57.226, -0.000, 0.000]
```

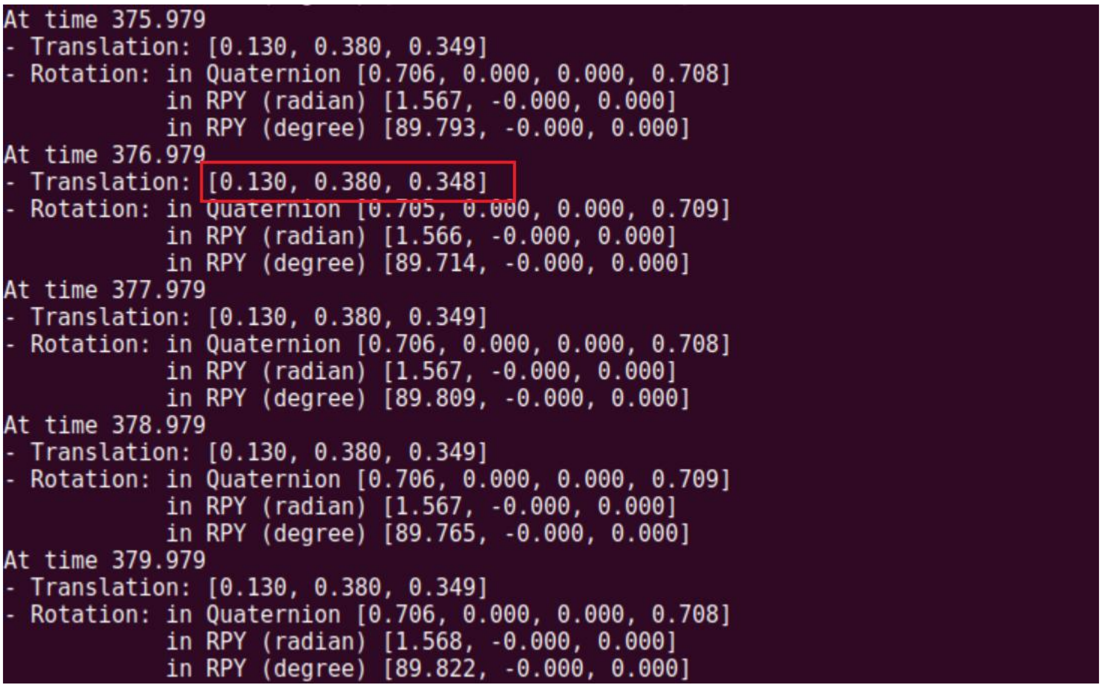
2. 完善 arm_kinematics.py

```python
# ==================== fk 子命令 ====================
```

```python
def cmd_fk(args):
    q1 = float(args[0]) if len(args) > 0 else 0.8
    q2 = float(args[1]) if len(args) > 1 else -0.9
    q1_limited, q2_limited = False, False
    # 限位判断
    if q1 < Q1_LIMITS[0]:
        q1 = Q1_LIMITS[0]
        q1_limited = True
    elif q1 > Q1_LIMITS[1]:
        q1 = Q1_LIMITS[1]
        q1_limited = True

    if q2 < Q2_LIMITS[0]:
        q2 = Q2_LIMITS[0]
        q2_limited = True
    elif q2 > Q2_LIMITS[1]:
        q2 = Q2_LIMITS[1]
        q2_limited = True

    s, d = fk_2r(q1, q2)
    x, y, z = to_torso(s, d)

    print("=" * 55)
    print(f"输入: q1 = {q1:.4f} rad ({math.degrees(q1):.1f} deg) {'限位'
if q1_limited else ''}")
    print(f"      q2 = {q2:.4f} rad ({math.degrees(q2):.1f} deg) {'限位'
if q2_limited else ''}")
```

运行结果:



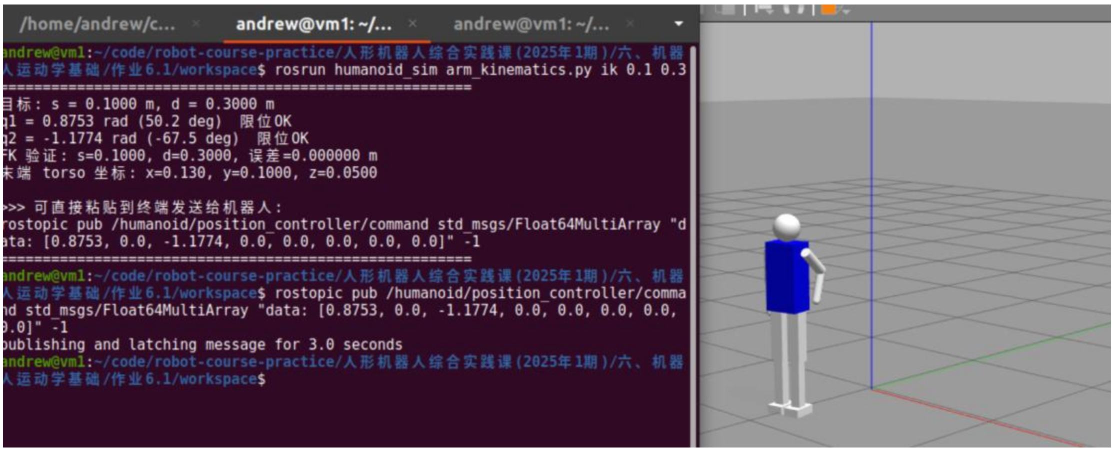


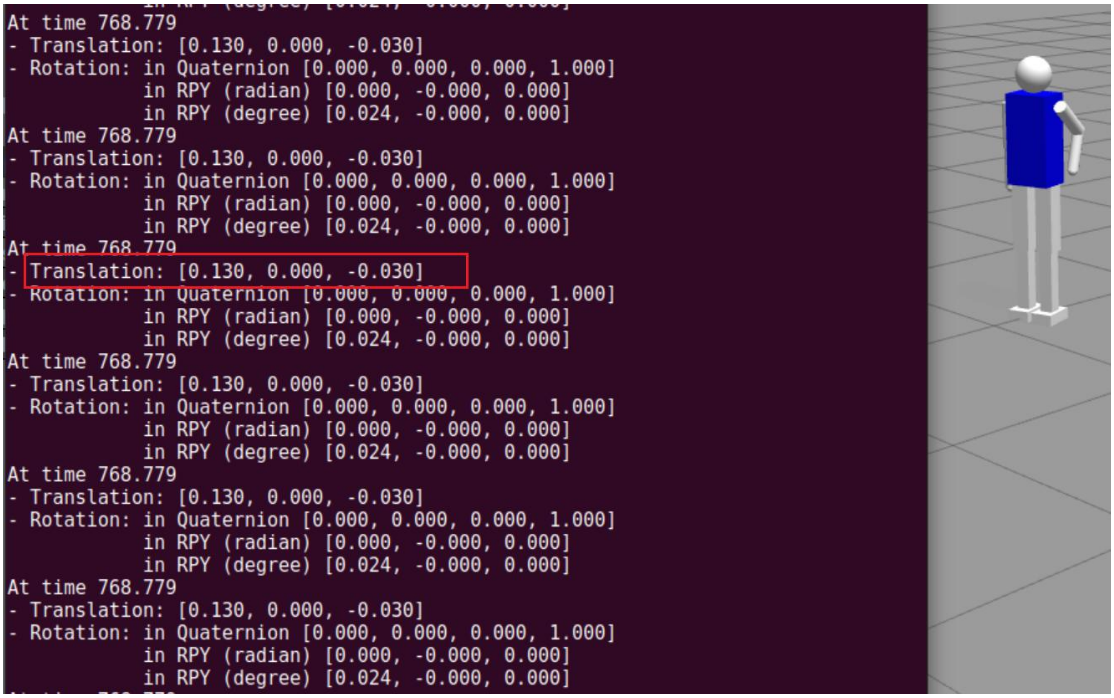
1.2 逆运动学闭环验证

（1）测试结果表格

序号	s	d	脚本预计算	tf_echo /torso /left_hand 输出
1	0.1	0.3	x=0.130, y=0.1000, z=0.0500	0.130, 0.100, 0.050
2	0.2	0.2	x=0.130, y=0.2000, z=0.1500	0.130, 0.200, 0.150
3	0.2	0.3	x=0.130, y=0.2000, z=0.0500	0.130, 0.199, 0.050

测试截图:

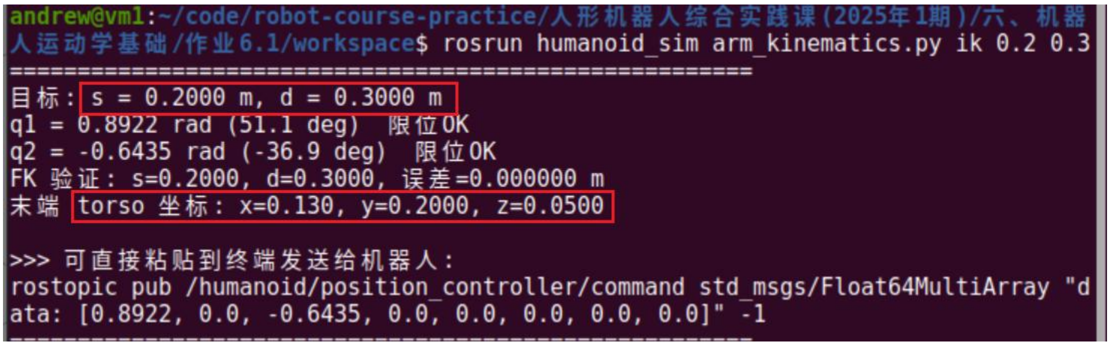




```
At time 768.779
- Translation: [0.130, 0.000, -0.030]
- Rotation: in Quaternion [0.000, 0.000, 0.000, 1.000]
            in RPY (radian) [0.000, -0.000, 0.000]
            in RPY (degree) [0.024, -0.000, 0.000]
At time 768.779
- Translation: [0.130, 0.000, -0.030]
- Rotation: in Quaternion [0.000, 0.000, 0.000, 1.000]
            in RPY (radian) [0.000, -0.000, 0.000]
            in RPY (degree) [0.024, -0.000, 0.000]
At time 768.779
- Translation: [0.130, 0.000, -0.030]
- Rotation: in Quaternion [0.000, 0.000, 0.000, 1.000]
            in RPY (radian) [0.000, -0.000, 0.000]
            in RPY (degree) [0.024, -0.000, 0.000]
At time 768.779
- Translation: [0.130, 0.000, -0.030]
- Rotation: in Quaternion [0.000, 0.000, 0.000, 1.000]
            in RPY (radian) [0.000, -0.000, 0.000]
            in RPY (degree) [0.024, -0.000, 0.000]
At time 768.779
- Translation: [0.130, 0.000, -0.030]
- Rotation: in Quaternion [0.000, 0.000, 0.000, 1.000]
            in RPY (radian) [0.000, -0.000, 0.000]
            in RPY (degree) [0.024, -0.000, 0.000]
At time 768.779
- Translation: [0.130, 0.000, -0.030]
- Rotation: in Quaternion [0.000, 0.000, 0.000, 1.000]
            in RPY (radian) [0.000, -0.000, 0.000]
            in RPY (degree) [0.024, -0.000, 0.000]
```

（2）思考题

ik 的输入参数为 s、d，位于肩部坐标系的 yz 二维平面内。肩部坐标系同 torso 坐标系需要通过 x、z 轴平移达到，也就导致输入的末端位置和 torso 中显示的并不一致。



```
andrew@vm1:~/code/robot-course-practice/人形机器人综合实践课(2025年1期)/六、机器人运动学基础/作业6.1/workspace$ rosrun humanoid_sim arm_kinematics.py ik 0.2 0.3
============================================================
目标: s = 0.2000 m, d = 0.3000 m
q1 = 0.8922 rad (51.1 deg)  限位OK
q2 = -0.6435 rad (-36.9 deg)  限位OK
FK 验证: s=0.2000, d=0.3000, 误差=0.000000 m
末端 torso 坐标: x=0.130, y=0.2000, z=0.0500

>>> 可直接粘贴到终端发送给机器人:
rostopic pub /humanoid/position_controller/command std_msgs/Float64MultiArray "data: [0.8922, 0.0, -0.6435, 0.0, 0.0, 0.0, 0.0, 0.0]" -1
```

解决思路：根据 urdf 文件找出肩部坐标系同 torso 坐标系的差异、变换矩阵，将(s, d)转换成 torso 坐标系下的值。

```python
SHOULDER_IN_TORSO = (0.13, 0.0, 0.35)
Q1_LIMITS = (-1.57, 1.57)
Q2_LIMITS = (-2.0, 0.0)
RMAX = L1 + L2
RMIN = abs(L1 - L2)


def fk_2r(q1, q2):
    """2R 平面 FK → (s, d)"""
    s = L1 * math.sin(q1) + L2 * math.sin(q1 + q2)
    d = L1 * math.cos(q1) + L2 * math.cos(q1 + q2)
    return s, d


def to_torso(s, d):
    """(s, d) → torso 坐标系 (x, y, z)"""
    return SHOULDER_IN_TORSO[0], s, SHOULDER_IN_TORSO[2] - d
```